Разработка модулей ядра Linux: Часть 6. Модули ядра vs пользовательские процессы

Олег Цилюрик
преподаватель тренингового отделения
Global Logic

03.05.2012

В этой статье описываются ключевые отличия между разработкой модулей ядра и разработкой программ для пользовательского пространства, также в ней приведен краткий обзор процессорных архитектур, поддерживаемых платформой Linux.

Введение

В предыдущей части статьи были показаны схемы взаимодействия всех составных частей, имеющих отношение к работе модулей ядра Linux. Теперь можно переходить к ответу на основной вопрос: в чём же заключаются отличия программы, выполняющейся как пользовательский процесс, от модуля ядра? Но сначала следует сказать несколько слов о различных типах архитектур, с которыми возможно придется столкнуть разработчику модулей ядра.

Другие типы архитектур

На любой другой (не Intel x86) архитектуре (аппаратной платформе, поддерживающей Linux и GCC, – их количество возросло в последнее время) системный вызов `syscall()` будет «доведен» до команды программного прерывания (вызова ядра), применяемой на данной платформе, например, `EMT` или `TRAP` и т.д.

Конкретный **вид и размер** таблицы системных вызовов зависит от процессорной архитектуры, под которую компилируется ядро. Естественно, эта таблица определена в ассемблерной части кода ядра, но даже **имена** и структура файлов при этом отличаются. Для подтверждения этого можно рассмотреть сравнительные определения двух схожих системных вызовов для двух различных архитектур (в исходных кодах ядра 2.6.37) — x86 и ARM:

© Copyright IBM Corporation 2012

Листинг 2. Таблицы системных вызовов для разных архитектур

```
$ cat /usr/src/linux-2.6.37.3/arch/x86/kernel/syscall_table_32.S | tail -n10
...
 .long sys_rt_tgsigqueueinfo /* 335 */
 .long sys_perf_event_open
 .long sys_recvmmsg
 .long sys_fanotify_init
 .long sys_fanotify_mark
 .long sys_prlimit64  /* 340 */
...
$ cat /usr/src/linux-2.6.37.3/arch/arm/kernel/calls.S | tail -n 20
...
/* 365 */ CALL(sys_recvmmsg)
  CALL(sys_accept4)
  CALL(sys_fanotify_init)
  CALL(sys_fanotify_mark)
  CALL(sys_prlimit64)
...
```

Последние вызовы в каждой из таблиц имеют одинаковое название: `sys_prlimit64` (а значит, можно предполагать, что и одинаковую семантику), но идентификатор вызова в архитектуре x86 равен **340**, а в ARM — **369**. Это говорит и о том, что некоторые системные вызовы, существующие в архитектуре ARM, отсутствуют в архитектуре x86.

И, если продолжать рассмотрение отдельных архитектур, то можно узнать, какие архитектуры поддерживаются данной версией ядра Linux:

Листинг 3. Архитектуры процессоров, поддерживаемые Linux

```
$ ls /usr/src/linux-2.6.37.3/arch
alpha  avr32  cris  h8300  m68k  microblaze  mn10300  powerpc  score  sparc  um  xtensa
arm  blackfin  frv  ia64  m32r  m68knommu  mips  parisc  s390  sh  tile  x86
$ ls /usr/src/linux-2.6.37.3/arch | wc -w
25
```

Как видно, на данный момент поддерживается 25 архитектур, кроме того, в некоторых случаях имя архитектуры — это «родовое» название, объединяющее множество отдельных технических реализаций, часто мало совместимых друг с другом. Самый характерный пример такого рода – это ARM- архитектура:

Листинг 4. Некоторые модели ARM-процессоров, поддерживаемые в Linux

```
$ ls /usr/src/linux-2.6.37.3/arch/arm

...              mach-ixp23xx     mach-omap2     mach-shark      plat-omap
...              mach-ixp4xx      mach-orion5x   mach-shmobile   plat-orion
mach-aaec2000    mach-kirkwood    mach-pnx4008   mach-spear3xx   plat-pxa
...
$ ls /usr/src/linux-2.6.37.3/arch/arm | wc -w
99
```

Даже если учесть, что некоторые файлы из этого каталога не являются архитектурными и имеют другое значение, то список из 90 типов архитектуры все равно впечатляет.

Отличия кодов процесса и ядра

Некоторые из различий уже достаточно очевидны и только требуют четкой формулировки, другие можно позаимствовать из литературы (они понятны и не требуют обоснований). Но оставшиеся различия уже не так просты, и с ними предстоит разобраться с помощью специально написанных тестовых примеров.

Во-первых, **самое важное отличие** для программиста, пишущего код модуля: ядро не имеет доступа к стандартным библиотекам языка C (как, собственно, и к любым другим библиотекам). Для этого есть несколько причин, но их обсуждение выходит за рамки статьи и не относится к ней. А как следствие, ядро оперирует со своим собственным набором API, **отличающимся** от POSIX API (отличающимся по набору функций, по их наименованиям, по их прототипам, параметрам, и т.д.). Это видно на примере идентичных по смыслу, но различающихся вызовов `printk()` и `printf()`. И если и будут иногда встречаться **якобы идентичные** функции (`sprint(), strlen(), strcat()` и многие другие), то это только внешняя **видимость совпадения**. Эти функции реализуют ту же функциональность, но это **дубликатная** реализация: подобный код реализуется и размещается в разных местах: для POSIX API в составе библиотек, а для модулей — в составе ядра.

Стоит отметить, что у этих двух эквивалентных реализаций будет и различная авторская принадлежность и время обновления. Реализация в составе библиотеки libc.so готовится сообществом GNU/FSF в комплексе проекта компилятора GCC, и новая версия библиотеки (и её заголовочные файлы в **/usr/include**) устанавливается, когда обновляется версия **компилятора**. А реализация той же функции в ядре создается разработчиками ядра Linux и будет обновляться при обновлении **ядра**, будь то из репозитария используемого дистрибутива или при самостоятельной сборке из исходных кодов. Эти обновления (**компилятора и ядра**), как очевидно, являются некоррелированными и несинхронизированными. Это неочевидная особенность, часто опускается из виду.

Косвенным следствием из сказанного будет то, что исходный код процесса и исходный код модуля в качестве каталогов для файлов определений (**.h**) по умолчанию (`#include <...>`) будут использовать совершенно разные источники: **/usr/lib/include** и **/lib/modules/`uname -r`/build/include**, соответственно. Но этот вопрос будет подробно рассмотрен ниже, в разделе, посвященном вариантам сборки модулей.

Следствием этой двойственности является то, что одной из основных трудностей при программировании модулей является **поиск и выбор** адекватных средств API из набора плохо документированных и регулярно меняющихся API ядра. Если для POSIX API существует большое количество качественных справочников, то по именам ядра (вызовам и структурам данных) таких руководств нет. А общая размерность имён ядра (в **/proc/kallsyms**) приближается к 100000, из которых до 10000 — это экспортируемые имена ядра.

Большинство механизмов ядра по своей функциональности сильно напоминают дуальные им и гораздо лучше известные механизмы POSIX, но специфика их исполнения в ядре (и ещё историческая преемственность) накладывается на реализацию, делая вызовы, отличающимися как по наименованию, так и по формату. Иногда очень помогает

отслеживание аналогичных вызовов из пространств пользователя и ядра. Примеры только некоторых таких совпадений собраны в таблице 1, и они говорят сами за себя.

Таблица. 1. Совпадающие вызовы в пространствах пользователя и ядра.

API процессов (POSIX)	API ядра
`strcpy(), strncpy(), strcat(), strncat(), strcmp (), strncmp(), strchr (), strlen(), strnlen(), strstr(), strrchr()`	`strcpy (), strncpy (), strcat(), strncat(), strcmp(), strncmp(), strchr(), strlen(), strnlen(), strstr(), strrchr()`
`printf()`	`printk()`
`execl(), execlp(), execle(), execv(), execvp(), execve()`	`call_usermodehelper()`
`malloc(), calloc(), alloca()`	`kmalloc(), vmalloc()`
`kill(), sigqueue()`	`send_sig()`
`open(), lseek(), read(), write(), close()`	`filp_open(), kernel_read(), kernel_write(), vfs_llseek(), vfs_read(), vfs_write(), filp_close()`
`pthread_create()`	`kernel_thread()`
`pthread_mutex_lock(), pthread_mutex_trylock(), pthread_mutex_unlock()`	`rt_mutex_lock(), rt_mutex_trylock(), rt_mutex_unlock()`

Второе отличие - это отсутствие защиты памяти. Если обычная программа предпринимает попытку некорректного обращения к памяти, то ядро аварийно завершит процесс, послав ему сигнал `SIGSEGV`. Если же само ядро предпримет попытку некорректного обращения к памяти, последствия могут быть более тяжелыми. К тому же ядро не использует замещение страниц: каждый байт, используемый в ядре – это один байт физической памяти.

В-третьих, в ядре нельзя использовать вычисления с плавающей точкой. Активизация режима вычислений с плавающей точкой требует (при переключении контекста) сохранения и восстановления регистров устройства поддержки вычислений с плавающей точкой (FPU). Вряд ли в модуле ядра могут понадобиться вещественные вычисления, но если такое и случится, то их нужно эмулировать через целочисленные вычисления (для этого существует множество библиотек, из которых может быть заимствован исходный код).

Последнее отличие – это фиксированный стек (область адресного пространства, в которой выделяются локальные переменные). Локальные переменные – это все переменные, объявленные внутри любого программного блока, начинающегося сразу за левой открывающей фигурной скобкой, и не имеющие ключевого слова static. Стек в режиме ядра ограничен по размеру и **не может быть** изменён. Поэтому в коде ядра нужно крайне осмотрительно использовать (или не использовать вообще) конструкции, сокращающие пространство стека: рекурсивные вызовы, передавать структуру в функцию в качестве параметра «по значению» или возвращать структуру из функции, объявление крупных локальных структур внутри функций и т.д. Обычно стек равен двум страницам памяти, что например для x86, соответствует 8 КБ для 32 разрядных систем и 16 КБ для 64 разрядных.

Заключение

Остается только закрепить полученную информацию подробным анализом выполнения системного вызова, что и будет сделано в следующей статье.

Об авторе

Олег Цилюрик



Олег Иванович Цилюрик, много лет был разработчиком программного обеспечения в крупных центрах разработки: ВНИИ РТ, НПО "Дельта", КБ ПМ. Последние годы работал над проектами в области промышленной автоматики, IP телефонии и коммуникаций. Автор нескольких книг. Преподаватель тренингового отделения международной софтверной компании Global Logic.

© Copyright IBM Corporation 2012
(www.ibm.com/legal/copytrade.shtml)
Торговые марки
(www.ibm.com/developerworks/ru/ibm/trademarks/)